

SGRSI

Sistema de Gestión de Recursos y Soporte Informático

Guía de estudio en 3 días

Proyecto de Bachillerato Tecnológico

HTML · CSS · JavaScript · PHP · MySQL

Versión con los errores de conexión corregidos
y el código PHP simplificado

Cómo usar esta guía

Esta guía está pensada para que en **tres días** puedas explicar y defender el proyecto completo: qué hace, cómo está armado, y por qué cada línea de código está donde está. No hace falta memorizar el código carácter por carácter; hace falta **entender el flujo** y poder reconstruirlo.

Día	Tema	Objetivo al terminar el día
Día 1	La base de datos y la conexión PHP	Sabés dibujar el modelo de datos de memoria y explicar cómo PHP se conecta a MySQL.
Día 2	Las APIs: login, sesiones y CRUD de usuarios	Podés explicar qué pasa desde que alguien aprieta "Ingresar" hasta que entra al sistema.
Día 3	El frontend en JavaScript y repaso general	Podés seguir el recorrido completo del dato: navegador → PHP → MySQL → navegador.

Método recomendado para cada día

1. **Leer** la sección del día completa, sin apurarte (unos 40 minutos).
2. **Abrir el archivo real** en tu editor y leerlo en paralelo con la explicación.
3. **Probarlo** en XAMPP: hacer que falle a propósito y ver el error.
4. **Cerrar todo** y escribir de memoria el archivo del día en una hoja.
5. **Responder** las preguntas de autoevaluación del final del día.

Mapa del proyecto: qué hace cada archivo

Archivo	Lenguaje	Para qué sirve
BD_SGRSI.sql	SQL	Crea la base de datos, las 6 tablas y carga los datos de prueba.
conexion_db.php	PHP	Se conecta a MySQL y define 3 funciones de ayuda que usan las demás APIs.
api_login.php	PHP	Inicia sesión, informa si hay sesión activa y cierra sesión.
api_usuarios.php	PHP	CRUD de usuarios: listar, crear y eliminar.
login.html + login.js	HTML/JS	Pantalla de acceso; manda el email y la contraseña a la API.
checklogin.js	JS	En cada página interna: verifica que haya sesión, muestra el nombre y maneja "Cerrar sesión".
index.html	HTML	Portada con los accesos a los seis módulos.
usuarios.js	JS	Módulo Usuarios. Es el único conectado a la base de datos.
Recursos.js	JS	Módulo Inventario (los datos viven en memoria).
prestamos.js	JS	Módulo Préstamos (memoria).
tickets.js	JS	Módulo Mesa de ayuda (memoria).
reportes.js	JS	Genera el resumen con totales (memoria).
historial.js	JS	Muestra tickets y reportes anteriores (memoria).
styles.css / login.css	CSS	Todo el diseño visual. No se tocó nada de esto.

Frase clave para la defensa oral

"El módulo de **Usuarios** y el **login** están conectados a MySQL a través de PHP. Los otros cuatro módulos funcionan con datos en memoria de JavaScript: son la maqueta funcional de la interfaz, y quedan preparados para conectarse siguiendo el mismo patrón que Usuarios."

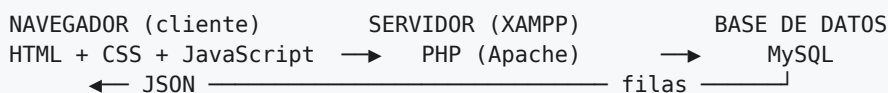
DÍA 1 — La base de datos y la conexión PHP

1.1 Qué es el SGRSI

El SGRSI es un sistema de gestión para el área de informática de un centro educativo. Resuelve cuatro necesidades concretas:

- **Inventario:** saber qué equipos existen, de qué tipo son y en qué estado están.
- **Préstamos:** registrar a quién se le prestó un equipo y cuándo lo devuelve.
- **Tickets:** una mesa de ayuda donde se reportan y cierran problemas técnicos.
- **Usuarios y roles:** quién puede entrar al sistema y con qué permisos.

La arquitectura es **cliente-servidor** clásica de tres capas:



1.2 La base de datos BD_SGRSI

Son **seis tablas**. Aprendí este esquema: es lo primero que preguntan.

Tabla	Clave primaria	Claves foráneas	Qué guarda
usuario	ci_usuario	—	nom, ape, email (único), contraseña (hash), rol
equipo	id_equipo	—	estado, numeroSerie, modelo, marca, tipo
servicio	id_servicio	ci_usuario	fechas, descripción del mantenimiento
prestamo	id_prestamo	ci_usuario, id_equipo	fechas, observación
ticket	id_ticket	ci_usuario	fechas, descripción, prioridad
historial	id_historial	id_equipo, id_ticket, id_servicio	descripción y acción realizada

Las relaciones, dichas en voz alta

- Un **usuario** puede tener muchos **préstamos**, muchos **tickets** y muchos **servicios** → relación **1 a N**.
- Un **equipo** puede estar en muchos **préstamos** a lo largo del tiempo → **1 a N**.
- **historial** es la tabla que cruza equipo + ticket + servicio: cada fila cuenta qué se le hizo a un equipo, por qué ticket y dentro de qué servicio.

Fragmento del SQL que tenés que poder escribir

```
create table usuario(  
    ci_usuario int primary key auto_increment,  
    nom varchar(20) not null,  
    ape varchar(20) not null,  
    email varchar(80) not null unique,  
    contrasena varchar(255) not null,  
    rol varchar(20) not null  
);  
  
create table prestamo(  
    id_prestamo int primary key auto_increment,  
    fechaCreacion varchar(20) not null,  
    fechaFin varchar(20) not null,  
    observacion varchar(50) not null,  
    ci_usuario int not null,  
    id_equipo int not null,  
    foreign key (ci_usuario) references usuario(ci_usuario),  
    foreign key (id_equipo) references equipo(id_equipo)  
);
```

Por qué cada detalle está ahí

- `primary key auto_increment` : identifica la fila sin repetirse; MySQL asigna el número solo.
- `unique` en email: impide dos cuentas con el mismo correo. Si intentás insertar uno repetido, MySQL devuelve el **error 1062** (esto aparece de nuevo en el Día 2).
- `varchar(255)` en contrasena: no guarda la contraseña, guarda el **hash**, que ocupa 60 caracteres. Se deja 255 por las dudas.
- `not null` : el campo es obligatorio.
- `foreign key` : garantiza que no exista un préstamo de un usuario que no existe. Es la **integridad referencial**.

Punto débil que te pueden marcar (tené la respuesta lista)

Las fechas están guardadas como `varchar(20)` y no como `DATE` . Respuesta honesta:
"Funciona, pero lo correcto sería `DATE` , porque permite ordenar cronológicamente, restar fechas y validar que sean fechas reales. Con `varchar`, '2026-13-45' se guardaría igual."

1.3 Poner el proyecto a andar en XAMPP

1. Abrir el **Panel de control de XAMPP** y arrancar **Apache** y **MySQL**.
2. Verificar que la carpeta del proyecto esté dentro de `C:\xampp\htdocs\` .
3. Entrar a `http://localhost/phpmyadmin` .
4. Pestaña **Importar** → elegir `BD_SGRSI.sql` → **Continuar**. Eso crea la base y carga los datos de prueba.
5. Abrir `http://localhost/proyectorevision/login.html` .

Muy importante

Siempre se entra por `http://localhost/...`, **nunca** haciendo doble clic en el archivo HTML. Si abris el archivo directo, la barra del navegador dice `file:///C:/...`, Apache no interviene, PHP no se ejecuta y **todas las llamadas al servidor fallan**. Es la causa número uno de "no me anda nada".

Usuarios de prueba

Email	Contraseña	Rol
juan.perez@utu.edu.uy	Perez123!	Administrador
ana.martinez@utu.edu.uy	Marti234!	Soporte Técnico
maria.garcia@utu.edu.uy	Garcia456!	Docente

1.4 conexion_db.php explicado línea por línea

Es el archivo más importante del backend: **todas** las APIs empiezan cargándolo con `require_once`. Si este archivo falla, falla el sistema entero.

Bloque 1 — la configuración

```
<?php
mysqli_report(MYSQLI_REPORT_OFF);

$DB_HOST = "localhost";
$DB_USER = "root";
$DB_PASS = "1234";
$DB_NAME = "BD_SGRSI";
```

- `mysqli_report(MYSQLI_REPORT_OFF)` : le pide a PHP que **no lance excepciones** cuando algo de MySQL falla. Así los errores se revisan con un simple `if`, que es más fácil de leer y de explicar que un `try/catch`.
- Las cuatro variables son los datos de acceso. En XAMPP el usuario suele ser `root`; la contraseña depende de cómo esté configurado tu MySQL. **Si tu XAMPP no tiene contraseña, poné "" vacío.**

Bloque 2 — la función conectar()

```
function conectar() {
    global $DB_HOST, $DB_USER, $DB_PASS, $DB_NAME;

    $conn = @new mysqli($DB_HOST, $DB_USER, $DB_PASS, $DB_NAME);

    if ($conn->connect_errno) {
        responderJson(["error" => "No se pudo conectar a la base de datos"], 500);
    }

    $conn->set_charset("utf8mb4");
    return $conn;
}
```

- `global` : dentro de una función, PHP no ve las variables de afuera. `global` le da acceso a las cuatro de arriba.

- `new mysqli(...)` : crea el objeto de conexión. Es la **extensión mysqli** ("MySQL improved"), la forma orientada a objetos de hablar con MySQL desde PHP.
- La **arroba** `@` silencia el warning que PHP imprimiría en HTML. Sin ella, ese warning se mezclaría con el JSON y el navegador no podría leer la respuesta.
- `connect_errno` : vale 0 si conectó bien, y un número distinto de 0 si falló. Si falló, cortamos con un JSON de error y el código **500**.
- `set_charset("utf8mb4")` : hace que las tildes, la ñ y los símbolos se guarden y se lean bien. Sin esto aparecen los famosos "María".

Bloque 3 — responderJson() y leerJson()

```
function responderJson($datos, $codigo = 200) {
    http_response_code($codigo);
    header("Content-Type: application/json; charset=utf-8");
    echo json_encode($datos);
    exit;
}

function leerJson() {
    $body = json_decode(file_get_contents("php://input"), true);

    if (is_array($body)) {
        return $body;
    }
    return $_POST;
}
```

- `responderJson` hace cuatro cosas: fija el **código HTTP**, avisa que lo que viene es JSON, convierte el array de PHP a texto JSON con `json_encode`, y `exit` corta el programa ahí mismo. Ese `exit` es el truco que hace que el resto del archivo funcione como una cadena de `if` sin `else`.
- `$codigo = 200` es un **parámetro por defecto**: si no se pasa, vale 200 (éxito).
- `leerJson` lee el cuerpo crudo de la petición (`php://input`) porque `fetch` manda JSON, y `$_POST` solo se llena con formularios tradicionales. `json_decode(..., true)` lo convierte en array asociativo de PHP. Si no vino JSON, cae a `$_POST` para no romper.

El error de conexión que tenía el proyecto (memorizá este)

El archivo tenía esta línea suelta, **sin las dos barras de comentario**:

```
Devuelve la conexión mysqli lista para usar
function conectar() { ...
```

PHP intentaba interpretar ese texto como código y tiraba un **Parse error**. Como `conexion_db.php` se carga con `require_once` desde las dos APIs, **ninguna** API respondía: el navegador recibía HTML de error en vez de JSON, y el `catch` del JavaScript mostraba "Error de conexión con el servidor". La corrección fue convertir esa línea en comentario (`//`).

Lección: cuando el frontend dice "error de conexión", el problema casi nunca está en el JavaScript. Abrí la pestaña **Red (Network)** del navegador y mirá la respuesta real del `.php`.

Por qué `conexion_db.php` no lleva `?>` al final

Si después de la etiqueta de cierre queda un espacio o un salto de línea, PHP lo envía al navegador **antes** del JSON. El JSON deja de ser válido y `res.json()` falla en el JavaScript. Por eso, en archivos que solo contienen PHP, la etiqueta de cierre se omite a propósito.

Autoevaluación del Día 1

1. ¿Cuántas tablas tiene la base y cuál es la clave primaria de cada una?
2. ¿Qué pasa si intento borrar un usuario que tiene préstamos registrados? ¿Por qué?
3. ¿Para qué sirve `set_charset("utf8mb4")` ?
4. ¿Qué hace `exit` dentro de `responderJson` y por qué es importante?
5. Cerré la guía y escribí de memoria la tabla `usuario` completa en SQL.

DÍA 2 — Las APIs: login, sesiones y CRUD

2.1 Conceptos que hay que tener claros antes

Qué es una API en este proyecto

Una **API** es un archivo PHP al que el JavaScript le pide o le manda datos, y que responde en **JSON** en lugar de HTML. El navegador nunca ve la base de datos: le pide todo a la API.

Los métodos HTTP

Método	Se usa para	En este proyecto
GET	Pedir información	Listar usuarios; preguntar si hay sesión activa; cerrar sesión.
POST	Crear algo nuevo / enviar datos sensibles	Iniciar sesión; crear un usuario.
DELETE	Borrar	Eliminar un usuario.

En PHP el método se lee con `$_SERVER["REQUEST_METHOD"]`. Por eso las dos APIs tienen la estructura `if ($metodo === "GET") { ... }, if ($metodo === "POST") { ... }`, etc.

Los códigos de estado HTTP que usa el proyecto

Código	Significado	Cuándo lo devuelve el SGRSI
200	OK	Todo salió bien.
201	Creado	Se insertó un usuario nuevo.
400	Petición incorrecta	Faltan campos, o el correo no tiene formato válido.
401	No autorizado	Email o contraseña incorrectos.
404	No encontrado	Se quiso borrar un usuario que no existe.
405	Método no permitido	Llegó un PUT o un método que la API no maneja.
409	Conflicto	El correo ya está registrado, o el usuario tiene registros asociados.
500	Error del servidor	No se pudo conectar a la base de datos.

2.2 api_login.php

Un solo archivo atiende tres cosas distintas, según el método y el parámetro `accion`:

Petición	Qué hace
POST api_login.php	Inicia sesión.
GET api_login.php	Responde si hay alguien logueado.
GET api_login.php?accion=cerrar	Cierra la sesión.

El encabezado

```
session_start();
require_once "conexion_db.php";

$metodo = $_SERVER["REQUEST_METHOD"];
```

- `session_start()` tiene que ir **antes de cualquier salida**. Arranca o retoma la sesión del usuario. PHP guarda un identificador en una cookie (`PHPSESSID`) y los datos quedan en el servidor.
- `require_once` carga la conexión una sola vez, aunque se pidiera dos veces.

POST — los cinco pasos del login

```
if ($metodo === "POST") {

    // 1) Leer lo que mandó el formulario
    $datos      = leerJson();
    $email      = trim($datos["email"] ?? "");
    $contrasena = $datos["contrasena"] ?? "";

    // 2) Validar
    if ($email === "" || $contrasena === "") {
        responderJson(["error" => "Email y contraseña requeridos"], 400);
    }

    // 3) Buscar el usuario por email
    $conn = conectar();
    $stmt = $conn->prepare("SELECT ci_usuario, nom, ape, email, contrasena, rol
                           FROM usuario
                           WHERE email = ?");
    $stmt->bind_param("s", $email);
    $stmt->execute();
    $usuario = $stmt->get_result()->fetch_assoc();
    $stmt->close();
    $conn->close();
}
```

- `trim()` saca los espacios de los costados: si alguien escribe " `juan@utu.edu.uy` ", igual entra.
- `??` es el **operador de fusión de null**: `$datos["email"] ?? ""` significa "si existe, usalo; si no, usá cadena vacía". Evita el warning "undefined index".
- La contraseña **no** lleva `trim`: un espacio puede ser parte legítima de la contraseña.
- `fetch_assoc()` trae **una** fila como array asociativo (`$usuario["nom"]`). Si no encontró nada, devuelve `null`.

La comparación de la contraseña

```
$hash = $usuario["contrasena"] ?? "";
if (substr($hash, 0, 4) === '$2b$') {
    $hash = '$2y$' . substr($hash, 4);
}

if (!$usuario || !password_verify($contrasena, $hash)) {
    responderJson(["error" => "Credenciales inválidas"], 401);
}
```

Cómo funcionan las contraseñas (tema estrella de la defensa)

- En la base **nunca** se guarda la contraseña real. Se guarda un **hash**: el resultado de pasarla por una función matemática de **una sola dirección**. Del hash no se puede volver a la contraseña.
- `password_hash()` genera el hash; `password_verify()` compara una contraseña escrita contra un hash guardado y devuelve `true` o `false`.
- El algoritmo es **bcrypt**, que además incluye una **sal** (salt) aleatoria: dos usuarios con la misma contraseña tienen hashes distintos.
- **El detalle del \$2b\$** : los hashes del archivo SQL fueron generados con la librería bcrypt de Node.js, que los etiqueta `$2b$`. PHP escribe `$2y$`. Es **exactamente el mismo algoritmo**, solo cambia la etiqueta, pero `password_verify` de PHP no reconoce `$2b$` y devolvería `false` siempre. Por eso se reemplaza el prefijo antes de comparar. Sin esas tres líneas, **ningún usuario de prueba podría entrar**.

Notá que el mensaje de error es siempre el mismo: "**Credenciales inválidas**", tanto si el email no existe como si la contraseña está mal. Es a propósito: si dijera "ese email no existe", un atacante podría averiguar qué correos están registrados.

Guardar la sesión

```
$_SESSION["usuario"] = [
    "ci_usuario" => $usuario["ci_usuario"],
    "nom"        => $usuario["nom"],
    "ape"        => $usuario["ape"],
    "email"      => $usuario["email"],
    "rol"        => $usuario["rol"]
];

responderJson([
    "mensaje" => "Inicio de sesión exitoso",
    "usuario" => $_SESSION["usuario"]
]);
}
```

`$_SESSION` es un array **superglobal**: está disponible en cualquier archivo PHP que haya hecho `session_start()`. Fijate que **el hash de la contraseña no se guarda ahí**, y por lo tanto tampoco viaja al navegador.

GET — estado de la sesión y cierre

```
if ($metodo === "GET") {  
    if (($_GET["accion"] ?? "") === "cerrar") {  
        $_SESSION = [];           // vacía los datos  
        session_destroy();        // destruye la sesión  
        responderJson(["mensaje" => "Sesión cerrada"]);  
    }  
  
    if (isset($_SESSION["usuario"])) {  
        responderJson([  
            "autenticado" => true,  
            "usuario"      => $_SESSION["usuario"]  
        ]);  
    }  
  
    responderJson(["autenticado" => false]);  
}  
  
responderJson(["error" => "Método no permitido"], 405);
```

Cerrar sesión son dos pasos: vaciar el array y destruir la sesión en el servidor. La última línea solo se alcanza si el método no fue ni GET ni POST, gracias a los `exit` de `responderJson`.

2.3 api_usuarios.php — el CRUD

CRUD = Create, Read, Update, Delete. Este proyecto implementa **Create, Read y Delete**; el Update quedó pendiente (decilo así si te preguntan, es una respuesta correcta y honesta).

GET — listar

```
if ($metodo === "GET") {  
    $sql = "SELECT ci_usuario, nom, ape, email, rol  
            FROM usuario  
            ORDER BY ci_usuario DESC";  
  
    $resultado = $conn->query($sql);  
    responderJson($resultado->fetch_all(MYSQLI_ASSOC));  
}
```

- Se usa `query()` y no `prepare()` porque la consulta **no recibe ningún dato del usuario**: no hay nada que inyectar.
- El `SELECT` nombra las columnas una por una y **deja afuera contraseña**. Nunca se envían hashes al navegador.
- `ORDER BY ci_usuario DESC`: los últimos creados aparecen arriba.
- `fetch_all(MYSQLI_ASSOC)` trae **todas** las filas de una vez, como array de arrays asociativos.

POST — crear

```
$datos = leerJson();
$nombre = trim($datos["nombre"] ?? "");
$correo = trim($datos["correo"] ?? "");
$rol = trim($datos["rol"] ?? "");

if ($nombre === "" || $correo === "" || $rol === "") {
    responderJson(["error" => "Faltan campos obligatorios"], 400);
}
if (!filter_var($correo, FILTER_VALIDATE_EMAIL)) {
    responderJson(["error" => "El correo no es válido"], 400);
}

$partes = explode(" ", $nombre, 2);
$nom = $partes[0];
$ape = $partes[1] ?? "";

$pass = password_hash("Default123!", PASSWORD_DEFAULT);

$stmt = $conn->prepare("INSERT INTO usuario (nom, ape, email, contrasena, rol)
VALUES (?, ?, ?, ?, ?)");
$stmt->bind_param("sssss", $nom, $ape, $correo, $pass, $rol);
$ok = $stmt->execute();

if (!$ok) {
    if ($conn->errno === 1062) {
        responderJson(["error" => "Ese correo ya está registrado"], 409);
    }
    responderJson(["error" => "No se pudo crear el usuario"], 500);
}

responderJson(["mensaje" => "Usuario creado", "id" => $conn->insert_id], 201);
```

- **Validación doble:** el HTML ya valida con `required` y `type="email"`, pero eso se puede saltar (por ejemplo con las herramientas de desarrollador). **La validación del servidor es la que realmente protege.** Frase de defensa: *"la validación del cliente es comodidad, la del servidor es seguridad"*.
- `filter_var(..., FILTER_VALIDATE_EMAIL)`: función nativa de PHP que verifica el formato del correo.
- `explode(" ", $nombre, 2)`: parte el texto en un máximo de **2** pedazos. "María del Carmen González" → `nom = "María"`, `ape = "del Carmen González"`. Se hace así porque el formulario pide un solo campo pero la tabla tiene `nom` y `ape` separados.
- La contraseña inicial es fija (`Default123!`) pero se guarda **hasheada**, igual que las demás.
- **El error 1062** es el código de MySQL para "clave duplicada". Como la columna `email` es `UNIQUE`, insertar un correo repetido lo dispara; se traduce a un 409 con mensaje claro en castellano.
- `$conn->insert_id`: el id que MySQL le asignó automáticamente a la fila nueva.

Consultas preparadas y SQL injection (pregunta segura del tribunal)

Sin consulta preparada, uno concatenaría el dato dentro del SQL:

```
"SELECT * FROM usuario WHERE email = '' . $email . ''"
```

Si alguien escribe como email ' OR '1'='1', la consulta que llega a MySQL es:

```
SELECT * FROM usuario WHERE email = '' OR '1'='1'
```

...y devuelve **todos** los usuarios. Eso es **SQL injection**.

Con `prepare()` + `bind_param()` el SQL viaja primero **con signos de pregunta** como marcadores, y los datos se mandan aparte. MySQL ya compiló la estructura de la consulta, así que lo que llega después **solo puede ser un valor**, nunca instrucciones.

Las letras de `bind_param` declaran el tipo de cada dato: "s" = string, "i" = entero, "d" = decimal. En "sssss" hay cinco letras porque hay cinco signos de pregunta.

DELETE — eliminar

```
if ($metodo === "DELETE") {  
    $id = (int)($_GET["id"] ?? 0);  
  
    if ($id <= 0) {  
        responderJson(["error" => "ID inválido"], 400);  
    }  
  
    $stmt = $conn->prepare("DELETE FROM usuario WHERE ci_usuario = ?");  
    $stmt->bind_param("i", $id);  
    $ok = $stmt->execute();  
  
    if (!$ok) {  
        responderJson(["error" => "No se puede eliminar: tiene registros asociados"], 409);  
    }  
  
    if ($stmt->affected_rows === 0) {  
        responderJson(["error" => "El usuario no existe"], 404);  
    }  
  
    responderJson(["mensaje" => "Usuario eliminado"]);  
}
```

- `(int)` es un **casteo**: fuerza el valor a número entero. Si llega `?id=abc`, se convierte en 0 y se rechaza.
- **Dos situaciones distintas, dos respuestas distintas**: si `execute()` devuelve `false`, la eliminación fue **bloqueada por una clave foránea** (el usuario tiene préstamos, tickets o servicios) → 409. Si se ejecutó pero `affected_rows` es 0, la consulta funcionó pero **no había fila con ese id** → 404.

Autoevaluación del Día 2

1. Contá los cinco pasos del login, en orden, sin mirar.
2. ¿Por qué el hash `$2b$` hay que convertirlo a `$2y$`?
3. ¿Qué es SQL injection y cómo lo evita este proyecto?

4. Diferencia entre devolver 409 y devolver 404 al eliminar un usuario.
5. ¿Por qué el `SELECT` de listar no incluye la columna `contrasena`?
6. Escribí de memoria el bloque `prepare` + `bind_param` + `execute` del INSERT.

DÍA 3 — El frontend en JavaScript y repaso general

3.1 fetch, promesas y async/await

`fetch()` es la función del navegador que pide algo al servidor **sin recargar la página**. Como la respuesta tarda, no devuelve el dato: devuelve una **promesa** (la promesa de que el dato va a llegar). Hay dos formas de esperarla:

```
// Forma 1: .then()
fetch("api_login.php")
  .then(res => res.json())
  .then(data => { /* usar data */ });

// Forma 2: async / await (se lee de arriba abajo, como código normal)
const res = await fetch("api_login.php");
const data = await res.json();
```

- `await` solo puede usarse dentro de una función marcada como `async`.
- `res.json()` convierte el texto JSON en un objeto de JavaScript. **También devuelve una promesa**, por eso lleva su propio `await`.
- `res.ok` es `true` si el código HTTP está entre 200 y 299. Es lo que permite distinguir "salió bien" de "el servidor devolvió un 400/401/500".
- `try / catch` alrededor del `fetch` atrapa los errores de **red**: servidor apagado, ruta inexistente, respuesta que no es JSON válido.

3.2 login.js

```
const API_LOGIN = "api_login.php";

const formLogin = document.getElementById("loginForm");
const errorDiv = document.getElementById("loginError");

formLogin.addEventListener("submit", async function (e) {
  e.preventDefault();
  errorDiv.style.display = "none";

  const email = document.getElementById("email").value.trim();
  const contrasena = document.getElementById("contrasena").value;

  try {
    const res = await fetch(API_LOGIN, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ email, contrasena })
    });

    const data = await res.json();

    if (!res.ok) {
      mostrarError(data.error || "Error al iniciar sesión");
      return;
    }

    window.location.href = "index.html";
  } catch (err) {
    console.error(err);
    mostrarError("Error de conexión con el servidor");
  }
});
```

- **La ruta es relativa** ("api_login.php" , sin barra al principio). Antes apuntaba a `/Proyectorevision-main/...` y daba **404** si la carpeta se llamaba distinto. Con la ruta relativa, el proyecto funciona con cualquier nombre de carpeta dentro de `htdocs`.
- `e.preventDefault()` : impide que el formulario recargue la página, que es su comportamiento por defecto. **Sin esta línea el fetch no llega a completarse.**
- `headers: {"Content-Type": "application/json"}` : le avisa al servidor en qué formato va el cuerpo.
- `JSON.stringify({ email, contrasena })` : convierte el objeto de JS en texto JSON. Es la operación **inversa** de `json_decode` en PHP.
- `window.location.href = "index.html"` : redirige al inicio si el login salió bien.

3.3 checklogin.js — la protección de las páginas internas

```
const API_SESSION = "api_login.php";

fetch(API_SESSION)
  .then(res => res.json())
  .then(data => {
    if (!data.autenticado) {
      window.location.href = "login.html";
      return;
    }

    const info = document.getElementById("userInfo");
    if (info) {
      const u = data.usuario;
      info.textContent = u.nom + " " + u.ape + " - " + u.rol;
    }
  });

const btnCerrar = document.getElementById("btnCerrarSesion");

if (btnCerrar) {
  btnCerrar.addEventListener("click", function (e) {
    e.preventDefault();
    fetch(API_SESSION + "?accion=cerrar")
      .finally(() => { window.location.href = "login.html"; });
  });
}
```

Este script está incluido en **todas** las páginas internas. Al cargar, le pregunta al servidor si hay sesión: si no la hay, devuelve al login; si la hay, muestra el nombre y el rol en el encabezado. El `if (btnCerrar)` evita un error si en alguna página no existe ese botón.

Límite del que conviene hablar antes de que te lo señalen

Esta protección es del **lado del cliente**: sirve para la navegación normal, pero alguien podría llamar a `api_usuarios.php` directamente sin estar logueado. La mejora sería agregar al principio de cada API:

```
if (!isset($_SESSION["usuario"])) {
  responderJson(["error" => "No autorizado"], 401);
}
```

Mostrar que conocés el límite y la solución vale más que fingir que no existe.

3.4 usuarios.js — el módulo conectado a la base

Tiene cuatro funciones. Es el archivo que mejor conviene saber, porque es **el ejemplo completo** de frontend hablando con backend.

Función	Qué hace
<code>cargarUsuarios()</code>	<code>fetch GET</code> → recibe el array → llama a <code>renderTablaUsuarios</code> .
<code>agregarUsuario()</code>	Valida los campos → <code>fetch POST</code> con el JSON → muestra el mensaje → recarga la tabla.
<code>eliminarUsuario(id)</code>	<code>confirm()</code> → <code>fetch DELETE</code> a <code>api_usuarios.php?id=...</code> → recarga la tabla.
<code>renderTablaUsuarios(u)</code>	Arma el HTML de la tabla como texto y lo mete con <code>innerHTML</code> .

```

async function cargarUsuarios() {
  try {
    const res = await fetch(API);
    const usuarios = await res.json();
    renderTablaUsuarios(usuarios);
  } catch (err) {
    mostrarMensaje(mensajeUsuario, "No se pudo conectar con el servidor.", "error");
  }
}

```

La última línea del archivo es `cargarUsuarios()`; : se ejecuta apenas carga la página, para que la tabla ya aparezca llena. Y después de cada alta o baja se la vuelve a llamar, así la pantalla siempre refleja lo que hay realmente en la base.

Cómo se arma la tabla

```

let html = "<table><thead><tr><th>#</th>...</tr></thead><tbody>";

usuarios.forEach((u, i) => {
  const nombreCompleto = (u.nom || "") + " " + (u.ape || "");
  html += "<tr>";
  html += "<td>" + (i + 1) + "</td>";
  html += "<td>" + nombreCompleto.trim() + "</td>";
  html += "<td>" + u.email + "</td>";
  html += "<td><span class='badge badge-info'>" + u.rol + "</span></td>";
  html += "<td><button onclick='eliminarUsuario(" + u.ci_usuario + ")'>Eliminar</button></td>";
  html += "</tr>";
});

html += "</tbody></table>";
contenedor.innerHTML = html;

```

Se construye todo el HTML en una variable de texto y recién al final se asigna a `innerHTML`. Es más rápido que ir agregando fila por fila al documento. Las clases (`badge`, `badge-info`) ya existen en `styles.css`: por eso el diseño sale bien sin tocar el CSS.

3.5 Los cuatro módulos que funcionan en memoria

Inventario, Préstamos, Tickets, Reportes e Historial **no usan PHP**: guardan los datos en arrays de JavaScript. Eso significa que **al recargar la página se pierden**. Es correcto decirlo así: son la maqueta funcional de la interfaz.

Módulo	Lógica que hay que poder explicar
Recursos.js Inventario	Valida el nombre (obligatorio, máximo 40 caracteres) y el tipo. Hace <code>equipos.push({...})</code> y redibuja la tabla. El estado se muestra con una <i>badge</i> verde o roja según <code>disponible</code> .
prestamos.js Préstamos	El <code><select></code> se llena solo con los equipos disponibles . Al registrar, marca <code>disponible = false</code> y el equipo desaparece de la lista. El botón "Devolver" lo vuelve a poner en <code>true</code> . Busca el equipo con <code>equipos.findIndex(e => e.id == idEquipo)</code> .
tickets.js Mesa de ayuda	Escucha el evento <code>submit</code> (no <code>click</code>) con <code>preventDefault()</code> . Guarda el ticket con estado "Abierto" y la fecha con <code>new Date().toLocaleDateString("es-UY")</code> . El color de la prioridad sale de un operador ternario anidado.
reportes.js Reportes	Recorre los arrays con <code>for</code> y va sumando contadores: disponibles/prestados, tickets abiertos/cerrados, préstamos activos/devueltos. Después arma las tarjetas y la tabla resumen.
historial.js Historial	Dos arrays de ejemplo y dos botones que cambian qué tabla se muestra en el mismo contenedor.

3.6 El recorrido completo del dato

Este es el **diagrama que tenés que poder dibujar en el pizarrón**. Ejemplo: dar de alta un usuario.

1. El usuario completa el formulario en `indexUsuarios.html` y aprieta el botón.
2. `usuarios.js` → `agregarUsuario()`
valida que no haya campos vacíos
`fetch("api_usuarios.php", { method:"POST",
body: JSON.stringify({nombre, correo, rol}) })`
3. Apache recibe la petición y ejecuta `api_usuarios.php`
4. `api_usuarios.php`
`require_once "conexion_db.php"` → `conectar()` → MySQL
`leerJson()` → toma los tres campos
valida en el servidor → si falla, responde 400
`explode()` → separa nom y ape
`password_hash()` → hashlea la contraseña por defecto
`prepare + bind_param + execute` → INSERT en la tabla usuario
5. MySQL guarda la fila y devuelve el id nuevo (`insert_id`)
6. `responderJson(["mensaje"=>"Usuario creado","id"=>...], 201)`
viaja de vuelta como texto JSON
7. `usuarios.js` → `res.ok` es true
muestra "Usuario agregado: ..."
limpia el formulario
llama a `cargarUsuarios()` → nuevo fetch GET → tabla actualizada

Errores que tenía el proyecto y cómo se corrigieron

Archivo	Problema	Corrección
conexion_db.php	Parse error fatal: una línea de comentario había perdido las <code>//</code> . Rompe las dos APIs a la vez.	Se comentó correctamente y se reescribió el archivo con comentarios claros.
conexion_db.php	El manejo de errores usaba <code>try/catch</code> con excepciones de <code>mysqli</code> , más difícil de explicar.	Se pasó a <code>mysqli_report(MYSQLI_REPORT_OFF)</code> + <code>if (\$conn->connect_errno)</code> .
api_usuarios.php	Los errores de MySQL se atrapaban con excepciones y el <code>affected_rows</code> se revisaba en el orden equivocado.	Se revisa el resultado de <code>execute()</code> con un <code>if</code> , y se separan claramente el 409 (clave foránea) del 404 (no existe).
index.html	Tenía dos etiquetas <code><html></code> , una dentro de la otra.	Quedó una sola: <code><html lang="es"></code> .
login.html, indexPrestamos.html, indexHistorial.html	La etiqueta <code><html></code> tenía el atributo <code>lang</code> repetido (<code>lang="es"</code> <code>lang="en"</code>).	Se dejó únicamente <code>lang="es"</code> .
login.html	El campo de contraseña tenía el atributo <code>required</code> escrito dos veces.	Se dejó uno solo.
Todas las páginas internas	Solo <code>index.html</code> verificaba la sesión; a las demás se entraba sin estar logueado y no tenían "Cerrar sesión".	Se agregó <code>checklogin.js</code> , el <code>div#userInfo</code> y el enlace de cerrar sesión en las seis páginas.
prestamos.js	Código muerto comentado y varios <code>console.log</code> de depuración dentro del bucle.	Se limpiaron; el <code><option></code> se arma con concatenación simple, igual que en el resto del proyecto.

El CSS no se tocó

`styles.css` , `login.css` y todo lo que hay en la carpeta `css/` quedaron **exactamente igual**. El sistema se ve idéntico a antes.

Preguntas típicas de defensa (con la respuesta corta)

1. ¿Qué hace tu sistema?

Gestiona el inventario de equipos, los préstamos, los tickets de soporte y los usuarios de un centro educativo, desde una interfaz web.

2. ¿Qué tecnologías usaste y por qué?

HTML y CSS para la interfaz, JavaScript para la interacción sin recargar la página, PHP como lenguaje del servidor y MySQL como base de datos. Todo corre sobre XAMPP, que ya trae Apache, PHP y MySQL integrados.

3. ¿Por qué hay archivos `api_` separados de los HTML?

Para separar responsabilidades: el HTML/JS es la presentación y las APIs son la lógica de datos. Así el mismo backend podría servir después a otro cliente, por ejemplo una app móvil.

4. ¿Cómo protegés las contraseñas?

No se guardan nunca en texto plano. Se guarda un hash bcrypt generado con `password_hash()`, que incluye sal aleatoria, y se compara con `password_verify()`. Del hash no se puede recuperar la contraseña original.

5. ¿Qué es una consulta preparada?

Una consulta que se manda a MySQL con signos de pregunta en lugar de los datos. MySQL compila primero la estructura y después recibe los valores por separado, así que un dato malicioso no puede convertirse en instrucción SQL. Es la defensa contra SQL injection.

6. ¿Cómo sabe el sistema que ya inicié sesión?

Con las sesiones de PHP. Al loguearse, los datos del usuario se guardan en `$_SESSION` del lado del servidor, y el navegador recibe una cookie con el identificador de sesión. Cada página interna llama a `api_login.php` por GET para preguntar si esa sesión sigue activa.

7. ¿Por qué usás JSON y no HTML como respuesta?

Porque JSON son solo datos: el JavaScript los convierte en objetos y decide cómo mostrarlos. Eso permite actualizar una parte de la página sin recargarla, y deja el backend independiente de la presentación.

8. Si borro un usuario que tiene préstamos, ¿qué pasa?

MySQL bloquea la eliminación por la clave foránea. La API detecta que el `execute()` falló y devuelve un 409 con el mensaje "No se puede eliminar: tiene registros asociados". Es integridad referencial funcionando.

9. ¿Validás los datos? ¿Dónde?

En los dos lados. En el HTML con `required` y `type="email"`, para dar aviso inmediato; y en el PHP de nuevo, porque la validación del cliente se puede saltar. La del servidor es la que realmente protege.

10. ¿Todos los módulos están conectados a la base?

No. Usuarios y el login trabajan contra MySQL. Inventario, Préstamos, Tickets, Reportes e Historial funcionan con datos en memoria: son la maqueta funcional de la interfaz y están preparados para conectarse siguiendo el mismo patrón de `api_usuarios.php`.

11. ¿Qué mejorarías si tuvieras más tiempo?

Conectar los cuatro módulos restantes a la base; agregar la operación de modificar (el "U" del CRUD); verificar la sesión también dentro de cada API; restringir acciones por rol; y cambiar las fechas de `varchar` a `DATE`.

12. ¿Qué es el error 1062?

El código de MySQL para clave duplicada. Aparece al intentar insertar un email que ya existe, porque esa columna es `UNIQUE`. La API lo traduce a un 409 con un mensaje entendible.

13. ¿Por qué las rutas del `fetch` son relativas?

Para que el proyecto funcione sin importar cómo se llame la carpeta dentro de `htdocs`. Con rutas absolutas, renombrar la carpeta rompía todo con un 404.

14. ¿Qué diferencia hay entre GET y POST acá?

GET pide información y sus parámetros van visibles en la URL. POST manda datos en el cuerpo de la petición y se usa para crear cosas o enviar información sensible, como la contraseña del login.

15. Si el sistema no responde, ¿cómo lo depurás?

Abro las herramientas de desarrollador, pestaña Red, y miro la respuesta real del archivo `.php`. Si es 404, la ruta está mal. Si es 500, hay un error de PHP o de base de datos. Si devuelve HTML en lugar de JSON, hay un error de sintaxis en el PHP. También reviso que Apache y MySQL estén encendidos en XAMPP.

Glosario

Término	Definición en una línea
API	Archivo del servidor que recibe pedidos y devuelve datos (acá, en JSON).
JSON	Formato de texto para intercambiar datos entre el navegador y el servidor.
fetch	Función de JavaScript que pide algo al servidor sin recargar la página.
Promesa	Objeto que representa un resultado que todavía no llegó.
async / await	Sintaxis para esperar promesas escribiendo el código de arriba abajo.
Hash	Resultado irreversible de aplicar una función matemática a un texto.
bcrypt	Algoritmo de hash pensado para contraseñas; incluye sal y es lento a propósito.
Sal (salt)	Valor aleatorio que se agrega antes de hashear para que dos claves iguales den hashes distintos.
Sesión	Espacio de memoria en el servidor asociado a un visitante, identificado por una cookie.
Superglobal	Variable de PHP disponible en cualquier archivo: <code>\$_SESSION</code> , <code>\$_POST</code> , <code>\$_GET</code> , <code>\$_SERVER</code> .
mysqli	Extensión de PHP para trabajar con MySQL.
Consulta preparada	Consulta con marcadores <code>?</code> a la que los datos se pasan por separado.
SQL injection	Ataque que inserta instrucciones SQL dentro de un campo de texto.
Clave primaria	Columna que identifica a cada fila de forma única.
Clave foránea	Columna que apunta a la clave primaria de otra tabla.
Integridad referencial	Garantía de que no existan referencias a filas inexistentes.
CRUD	Create, Read, Update, Delete: las cuatro operaciones básicas sobre datos.
Código HTTP	Número que resume el resultado de una petición (200, 401, 404, 500...).
localhost	Nombre que apunta a tu propia computadora funcionando como servidor.
htdocs	Carpeta de XAMPP donde Apache busca los sitios web.

Checklist final: ¿estoy listo?

Marcá cada punto solo cuando puedas hacerlo **sin mirar la guía**.

Base de datos

- ☐ Nombro las 6 tablas y la clave primaria de cada una.
- ☐ Explico las relaciones 1 a N entre usuario, equipo, préstamo y ticket.
- ☐ Escribo el `CREATE TABLE usuario` completo de memoria.
- ☐ Explico qué hace `UNIQUE` en `email` y con qué error se relaciona.
- ☐ Importo el `.sql` en phpMyAdmin sin ayuda.

PHP

- ☐ Explico las tres funciones de `conexion_db.php`.
- ☐ Cuento los cinco pasos del login en orden.
- ☐ Explico `password_hash` vs `password_verify` y el tema del `$2b$`.
- ☐ Explico qué es SQL injection y cómo lo evitan las consultas preparadas.
- ☐ Explico la diferencia entre 400, 401, 404, 409 y 500 en este proyecto.
- ☐ Explico por qué el archivo no lleva la etiqueta de cierre de PHP.

JavaScript

- ☐ Explico qué hace `fetch` y por qué devuelve una promesa.
- ☐ Explico por qué hace falta `e.preventDefault()`.
- ☐ Explico qué hace `JSON.stringify` y cuál es su equivalente en PHP.
- ☐ Explico qué revisa `checklogin.js` y cuál es su límite.
- ☐ Explico qué módulos usan la base y cuáles funcionan en memoria.

Sistema completo

- ☐ Dibujo el recorrido completo del alta de un usuario, de punta a punta.
- ☐ Levanto el proyecto en XAMPP e inicio sesión con un usuario de prueba.
- ☐ Doy de alta un usuario, lo veo en la tabla y lo confirmo en phpMyAdmin.
- ☐ Provoco un error a propósito (correo repetido) y explico la respuesta.
- ☐ Nombro tres mejoras futuras concretas.

Lo último, antes de la defensa

Repasá tres cosas: **(1)** el esquema de las 6 tablas, **(2)** los cinco pasos del login, **(3)** el recorrido completo del dato del punto 3.6. Con eso podés responder la mayoría de las preguntas, incluso las que no estén en esta guía. Y si te preguntan algo que el proyecto no hace, la mejor respuesta es reconocerlo y decir cómo lo harías.